



Design and Development of an IoT-Based HVAC Temperature and Humidity Monitoring System

EXERCISE NO – 08

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

8. Design and Development of an IoT-Based HVAC Temperature and Humidity Monitoring System

1. Project overview:

HVAC (Heating, Ventilation and Air Conditioning) systems are widely used in industries, offices, laboratories, hospitals, classrooms and commercial buildings to maintain comfortable and suitable environmental conditions.

The proposed project is an **IoT-Based HVAC Temperature and Humidity Monitoring System** that continuously measures the temperature and relative humidity of an HVAC environment. The measured data is collected using sensors connected to an **ESP32 microcontroller** and transmitted through Wi-Fi to an IoT dashboard.

The system allows users to monitor environmental conditions remotely using a mobile phone or computer. If the temperature or humidity exceeds the predefined limits, the system can generate an alert. The collected data can also be used to identify HVAC performance problems and support energy-management decisions.

2. Problem Statement:

Traditional HVAC systems generally depend on local thermostats or manual observation for monitoring environmental conditions. These methods provide limited information about the actual temperature and humidity distribution inside a room or industrial environment.

If the HVAC system operates continuously even when the required environmental conditions have already been achieved, unnecessary energy consumption can occur. Similarly, high humidity or abnormal temperature conditions may indicate poor ventilation, improper cooling, sensor problems, or HVAC malfunction.

The absence of continuous remote monitoring makes it difficult for operators to identify these problems quickly.

Therefore, there is a need for a low-cost IoT-based monitoring system that continuously measures temperature and humidity, sends the data to a cloud/mobile dashboard, stores historical information and provides alerts when abnormal conditions are detected.

The proposed system addresses this problem by integrating temperature and humidity sensors with an ESP32 Wi-Fi controller and an IoT platform.

3.Proposed Solution:

The proposed system uses a DHT22 temperature and humidity sensor connected to an ESP32 Wi-Fi microcontroller.

The sensor continuously measures:

- Temperature in °C
- Relative humidity in %

The ESP32 receives the sensor data and processes it. Through its built-in Wi-Fi capability, the ESP32 sends the measured data to an IoT cloud platform.

The data can be displayed on a dashboard containing:

- Current temperature
- Current humidity
- Temperature graph
- Humidity graph
- Maximum/minimum values
- Warning status
- Historical data

An alert can be generated if the measured values exceed predefined limits.

Example limits

Parameter	Normal Range	Warning condition
Temperature	22–26 °C	Above 26 °C
Humidity	40–60 % RH	Above 60 % RH

4.System Architecture:

The system architecture describes how the different components of the IoT-based HVAC monitoring system are connected and communicate with each other.

The temperature and humidity sensor continuously measures the environmental conditions of the HVAC area. The measured data is sent to the ESP32 microcontroller, which processes the sensor readings.

The ESP32 uses its built-in Wi-Fi connectivity to transmit the collected data to an IoT cloud platform. The cloud platform stores and processes the data, which can then be viewed through a web or mobile dashboard.

The user can monitor the temperature and humidity remotely. If the measured values exceed the predefined limits, the system can generate an alert or warning.

System flow:

Temperature & Humidity Sensor → ESP32 → Wi-Fi → IoT Cloud → Web/Mobile Dashboard → User

This architecture enables real-time monitoring, data storage, remote access, alert generation, and HVAC performance analysis.

5.Circuit Table

Component	Component Pin	ESP32 Pin
DHT22	VCC	3.3V
DHT22	GND	GND
DHT22	DATA	GPIO 4
OLED	VCC	3.3V
OLED	GND	GND
OLED	SDA	GPIO 21
OLED	SCL	GPIO 22
LED	Anode (+)	GPIO 2
LED	Cathode (-)	GND
Buzzer	Positive (+)	GPIO 5
Buzzer	Negative (-)	GND

6. Circuit Design:

The main controller used in this project is the ESP32 because it has built-in Wi-Fi and sufficient GPIO pins for sensor and display connections.

The DHT22 sensor is connected to GPIO 4 of the ESP32.

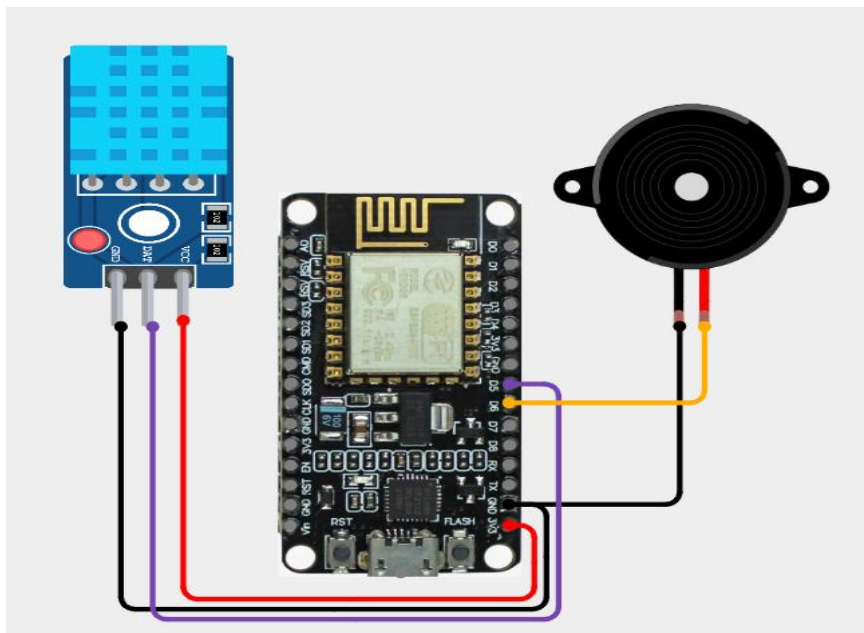
The sensor requires:

- **VCC → 3.3 V**
- **GND → GND**
- **DATA → GPIO 4**

An OLED display can be connected using the I²C interface:

- **SDA → GPIO 21**
- **SCL → GPIO 22**

A buzzer and LED can be used to indicate abnormal environmental conditions.



7. Algorithm:

Step 1: Start the system.

Step 2: Initialize ESP32.

Step 3: Initialize temperature and humidity sensor.

Step 4: Connect ESP32 to Wi-Fi.

Step 5: Read temperature from DHT22.

Step 6: Read humidity from DHT22.

Step 7: Check whether sensor readings are valid.

Step 8: Display temperature and humidity.

Step 9: Send readings to the IoT cloud platform.

Step 10: Compare measured values with predefined limits.

Step 11: If temperature or humidity exceeds the limit, activate warning/alert.

Step 12: Store the data for historical analysis.

Step 13: Wait for the next measurement interval.

Step 14: Repeat the process continuously.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>
#define WIFI_SSID    "K-APIL"
#define WIFI_PASS    "kapil.2333"
#define IO_USERNAME  "saravanakarthisr"
#define IO_KEY       "aio_idPg82y91fmNVXL0iJA71zKwH97q"
AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);
AdafruitIO_Feed *temperatureFeed =
  io.feed("temperature");
AdafruitIO_Feed *humidityFeed =
  io.feed("humidity");
AdafruitIO_Feed *refrigeratorStatusFeed =
  io.feed("refrigerator-status");
AdafruitIO_Feed *buzzerStatusFeed =
  io.feed("buzzer-status");
```

```

#define DHT_PIN 14
#define BUZZER_PIN 12
#define DHT_TYPE DHT11
DHT dht(DHT_PIN, DHT_TYPE);
float TEMP_MIN = 2.0;
float TEMP_MAX = 8.0;
float HUM_MIN = 30.0;
float HUM_MAX = 70.0;
#define BUZZER_ON HIGH
#define BUZZER_OFF LOW
void setup()
{
  Serial.begin(115200);
  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, BUZZER_OFF);
  dht.begin();
  Serial.println();
  Serial.println();
  Serial.println();
  Serial.println();
  Serial.println("Connecting to Adafruit IO...");
  io.connect();
  while (io.status() < AIO_CONNECTED)
  {
    Serial.print(".");
    delay(500);
  }
  Serial.println();
  Serial.println("Adafruit IO Connected!");
  Serial.println();
  Serial.print("Temperature Limit: ");
  Serial.print(TEMP_MIN);
  Serial.print(" - ");
  Serial.print(TEMP_MAX);
  Serial.println(" °C");
  Serial.print("Humidity Limit: ");
  Serial.print(HUM_MIN);
  Serial.print(" - ");
  Serial.print(HUM_MAX);
  Serial.println(" %");
  Serial.println();
}
void loop()
{

```

```

io.run();
float temperature = dht.readTemperature();
float humidity = dht.readHumidity();
if (isnan(temperature) || isnan(humidity))
{
    Serial.println("DHT11 SENSOR ERROR!");
    digitalWrite(BUZZER_PIN, BUZZER_ON);

    buzzerStatusFeed->save("ON");
    refrigeratorStatusFeed->save("ALERT");
    delay(2000);
    return;
}
Serial.println();
Serial.println();
Serial.print("Temperature : ");
Serial.print(temperature);
Serial.println(" °C");
Serial.print("Humidity  : ");
Serial.print(humidity);
Serial.println(" %");
bool temperatureAlert = false;
bool humidityAlert = false;
if (temperature > TEMP_MAX || temperature < TEMP_MIN)
{
    temperatureAlert = true;
}
if (humidity > HUM_MAX || humidity < HUM_MIN)
{
    humidityAlert = true;
}
if (temperatureAlert || humidityAlert)
{
    digitalWrite(BUZZER_PIN, BUZZER_ON);
    refrigeratorStatusFeed->save("ALERT");
    buzzerStatusFeed->save("ON");
    Serial.println("HVAC STATUS    : ALERT");
    Serial.println("BUZZER    : ON");
}
else
{
    digitalWrite(BUZZER_PIN, BUZZER_OFF);
    refrigeratorStatusFeed->save("NORMAL");
    buzzerStatusFeed->save("OFF");
}

```



```
Serial.println("HVAC STATUS    : NORMAL");  
Serial.println("BUZZER      : OFF");  
}  
temperatureFeed->save(temperature);  
humidityFeed->save(humidity);  
Serial.println("Data sent to Adafruit IO");  
Serial.println();  
delay(5000);  
}
```

9. System Working:

The system starts when power is supplied to the ESP32.

The DHT22 sensor continuously measures the environmental temperature and humidity near the HVAC outlet or inside the monitored room.

The ESP32 receives these values and processes them.

For example:

Temperature = 24.5 °C

Humidity = 52 %

Status = NORMAL

The ESP32 then sends the readings through Wi-Fi to the IoT cloud.

The dashboard can display the readings in real time.

If the temperature increases:

Temperature = 29 °C

Humidity = 65 %

Status = WARNING

the system activates the warning indicator and sends an alert to the user.

HVAC performance analysis

The collected data can be used to determine:

- Whether the room is maintaining the desired temperature.
- Whether humidity is too high.
- Whether the HVAC system is operating efficiently.
- Whether temperature changes occur frequently.
- Whether excessive HVAC operation may be occurring.

```

Adafruit IO Connected!

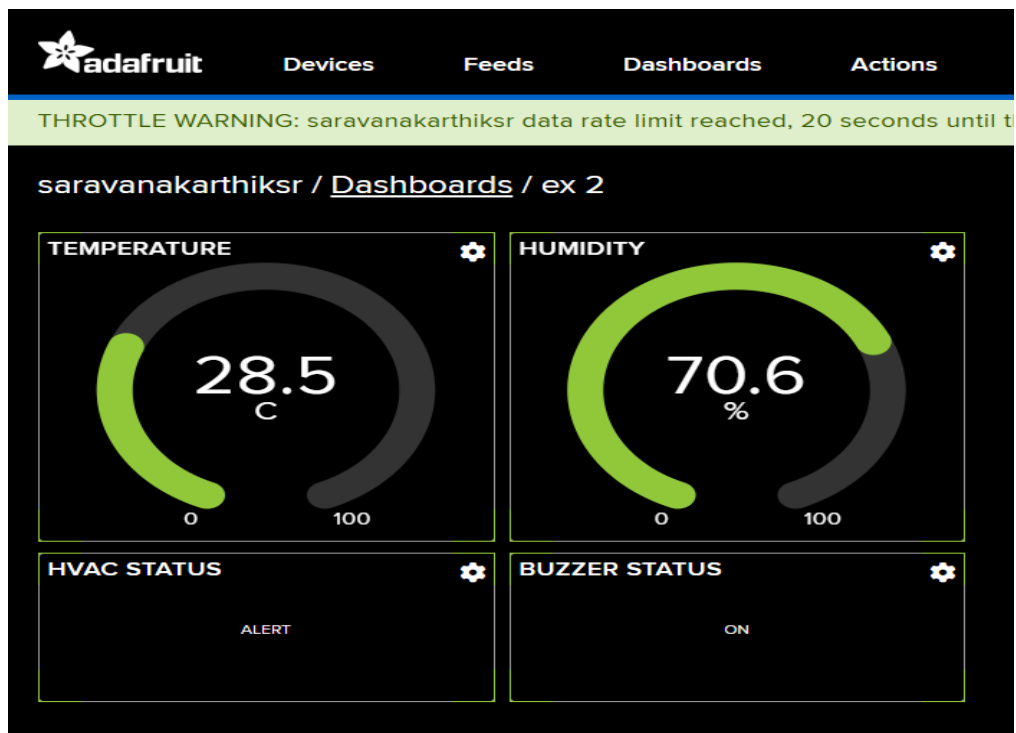
-----
Temperature Limit: 2.00 - 8.00 °C
Humidity Limit: 30.00 - 70.00 %
-----

===== REFRIGERATOR DATA =====
Temperature : 28.50 °C
Humidity    : 70.20 %
HVAC STATUS : ALERT
BUZZER      : ON
Data sent to Adafruit IO
=====

===== REFRIGERATOR DATA =====
Temperature : 28.50 °C
Humidity    : 70.60 %
HVAC STATUS : ALERT
BUZZER      : ON
Data sent to Adafruit IO
=====

===== REFRIGERATOR DATA =====
Temperature : 28.50 °C
Humidity    : 70.60 %
HVAC STATUS : ALERT
BUZZER      : ON
Data sent to Adafruit IO
=====

```



10. Bill of Materials:

COMPONENT	QUANTITY
ESP8266 NodeMCU	1
DHT22 Sensor	1
Relay Module	1
OLED/LCD Display	1
Buzzer	1
Breadboard	1
Jumper Wires	Required

11. Prototype Implementation:

The prototype can be implemented in the following stages.

Stage 1 – Sensor connection

Connect the DHT22 sensor to the ESP32 and verify temperature and humidity readings using the Arduino IDE Serial Monitor.

Stage 2 – Local display

Connect an OLED display to show:

HVAC MONITOR

Temperature: 24.5 °C

Humidity: 52 %

Status: NORMAL

Stage 3 – Wi-Fi connection

Program the ESP32 to connect to a Wi-Fi network.

Stage 4 – IoT connection

Connect the ESP32 to the selected IoT platform.

The dashboard can contain:

- Temperature gauge
- Humidity gauge
- Temperature graph

- Humidity graph
- Warning indicator

Stage 5 – Alert system

Set threshold values.

Example:

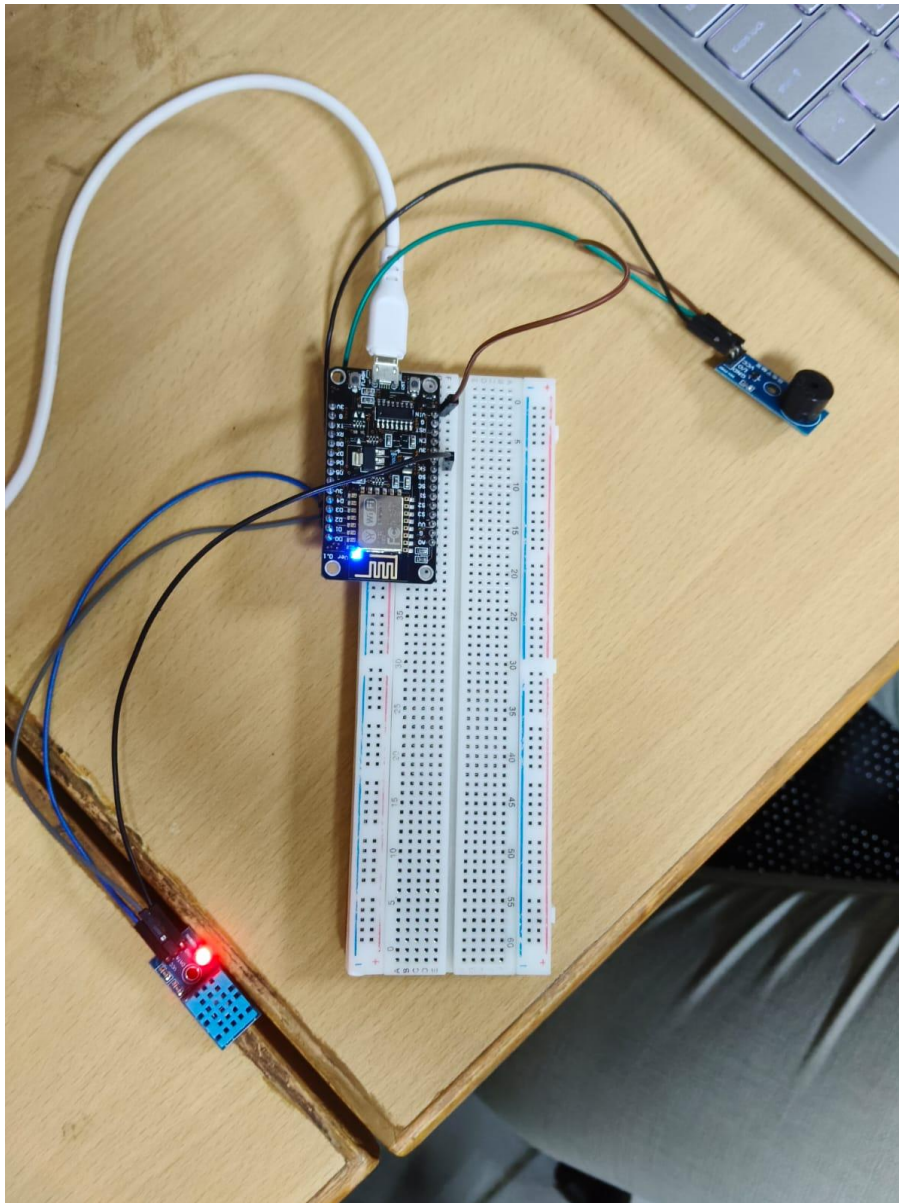
Temperature > 26 °C → Temperature Warning

Humidity > 60 % → Humidity Warning

Stage 6 – Prototype enclosure

Place the ESP32, sensor interface and display inside a suitable enclosure.

The sensor should be positioned where it can properly sense the room/HVAC environment and should not be placed directly against a hot or wet surface.



12.Results & Performance Analysis:

The developed IoT-Based HVAC Temperature and Humidity Monitoring System was successfully designed and implemented to continuously monitor the environmental conditions of an HVAC-controlled area. The system uses a DHT22 sensor to measure temperature and relative humidity, while the ESP32 microcontroller collects and processes the sensor readings.

The measured data is transmitted through Wi-Fi to the IoT platform, where it can be viewed remotely using a web or mobile dashboard. This allows the user to monitor the HVAC environment in real time without being physically present at the location.

The system also compares the measured temperature and humidity values with predefined limits. When the values remain within the required range, the system indicates normal operation. If the temperature or humidity exceeds the specified limit, the system generates a warning or alert. This helps the user identify abnormal environmental conditions at an early stage.

The continuous collection of temperature and humidity data also helps in analyzing HVAC performance over a period of time. For example, if the temperature remains high even when the HVAC system is operating, it may indicate insufficient cooling, poor ventilation, blocked filters, or other HVAC-related issues. Similarly, continuously high humidity levels may indicate inadequate dehumidification or ventilation problems.

The system reduces the need for manual monitoring and provides easy access to real-time and historical data. The collected information can be used to understand environmental patterns and support better HVAC operation and energy-management decisions.

Overall, the developed system provides a low-cost, reliable, and effective solution for continuous HVAC temperature and humidity monitoring. It can be further improved by adding energy-consumption sensors, automatic HVAC control, cloud-based data analytics, mobile notifications, and AI-based fault detection.